CSD201 PE INSTRUCTIONS

Read the instructions below carefully before you start coding.

Students are ONLY allowed to use:

- Materials on his/her computer (including JDK, NetBeans...).
- For distance learning: Google Meet, Hangout (for Exam Monitoring Purpose).

Follow the steps below to complete PE:

1. Create a folder to save given projects, e.g. CSD_given (1). Download the given materials to (1).
2. Steps to do question 1 (do the same for questions 2): Open NetBeans, open the given Q1 project, then edit the MyList.java file according to the requirements of the exam. (edit the file BSTree.java for Q2).
3. Before submission: **Clean and Build Project** (Shift+F11). If still error, try to rename or copy the project to other one.
4. **Submission**: browse and select the project folder then click the **Submit** button. **Do not submit** the un-edited given project.
5. **Do not use accented Vietnamese** when writing comments in programs.
6. **Do not add** new **import** statement(s) to given files.
7. Software tools must be used: **NetBeans IDE 8.x** and **Java JDK 1.8**. **If at least one of the above requirements is not followed, the exam will get ZERO.**

Troubleshooting:

If the given project (e.g. Q1) runs with error, you need to run "Clean and Build Project" (Shift+F11). If still error, try to rename or copy the project to other one, e.g. from Q1 to Q1X or Q1Y.

Do not pay attention to real meaning of objects, variables and their values in the questions below.

Question 1: (5 marks)

In this question, you need to complete some methods in the **MyList.java** file.

The class Phone with 3 data members: id, weight and price is given and you do not need to edit it. In this class, **id must be unique (there is no duplicated id in the list), weight>0, and price>0**. The MyList class is a linked list of Phone objects. The following methods must be completed:

- **void addLast(String xId, int xWeight, int xPrice)**.
- **void f1()** – Do not edit this method. Your task is to complete only the addLast(...) method above. The expected output in the file **f1.txt** should be the following:

```
(H,6,7) (C,4,5) (Y,5,8)
```

- **void f2()** – There is a given Phone object (named v) in this function. Suppose the list contains at least 3 elements. Write statements to insert v into the list so that **v will be the 3rd node** after insertion. The expected output in the file **f2.txt** should be the following:

```
(D,4,7) (E,9,2) (Q,7,6)
(D,4,7) (E,9,2) (New,8,9) (Q,7,6)
```

- **void f3()** – **Delete** the node holding the heaviest Phone (maximum weight). If there is more than one node satisfying the requirement, **delete the first of them**. The expected output in the file **f3.txt** should be the following:

```
(B,9,2) (D,12,6) (E,9,4) (R,12,7)
(B,9,2) (E,9,4) (R,12,7)
```

- **void f4()** – How many Phones in the list satisfy **price>5**? The expected output in the file **f4.txt** should be as follows:

```
(H,1,3) (I,3,7) (J,4,5) (K,5,6)
2
```

- **void f5()** – **Delete the first node**, then sort the elements **descending by weight** (assuming that all weights are different). The expected output in the file **f5.txt** should be the following:

```
(N,6,1) (A,9,3) (W,5,2) (S,7,5) (C,1,1)
(A,9,3) (S,7,5) (W,5,2) (C,1,1)
```

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*

Question 2: (5 marks)

In this question, you need to complete some methods in the **BSTree.java** file.

The class Dog with 3 data members: place, weight and color is given and you do not need to edit it. The BSTree class is a binary search tree of Dog objects. The variable **weight is the key** of the BSTree. **In this test, these values (weight) must be unique**. The following methods must be completed:

- **void insert(String xPlace, int xWeight, int xColor)**
- **void f1()** – Your task is to add statements in the **insert(...)** method so that any node with xPlace.charAt(0) == 'F' is not inserted to the tree. The expected output in file **f1.txt** should be as follows (line 1: Breadth first traversal, line 2: In-order traversal):

```
(X,5,2) (N,3,6) (M,9,5) (P,1,3) (R,4,8) (Q,8,7) (L,7,6)
(P,1,3) (N,3,6) (R,4,8) (X,5,2) (L,7,6) (Q,8,7) (M,9,5)
```

- **void f2()** – Perform pre-order traversal for the **whole** BST, but display only nodes with **color<8** to file **f2.txt**. **Hint**: Copy the function preOrder(...) to function preOrder2(...) and modify it. The expected output in the file **f2.txt** should be the following (line 1: Breadth first traversal, line 2: pre-order traversal):

```
(P,7,9) (D,4,3) (G,9,6) (L,1,8) (B,6,4) (X,8,9)
(D,4,3) (B,6,4) (G,9,6)
```

- **void f3()** – Perform a new traversal method to visit all nodes in the BST with the constraint: visit the nodes in the **descending order of weight**. The expected output in the file **f3.txt** must be the following:

```
(T,8,-5) (M,7,2) (P,6,3) (R,5,9) (S,4,1) (L,2,5)
```

- **void f4()** – Find the **right most node** of the tree. The expected output in the file **f4.txt** should be the following (line 1: pre-order traversal, line 2: the right most node):

```
(A,6,5) (B,2,7) (C,3,-8) (D,5,-3) (E,8,3) (G,7,4) (Q,9,1)
(Q,9,1)
```

- **void f5()** – How many **leaf nodes** are there in the BST? The expected output in the file **f5.txt** should be the following (line 1: in-order traversal, line 2: the amount of leaf nodes):

```
(H,2,6) (Q,3,1) (D,4,3) (B,6,-4) (M,7,9) (X,8,7) (G,9,-6)
3
```